Nordwise - Project Report

Student name: Hai Anh Pham

1. Project Overview

Nordwise is a website which aims to explore Finland municipality statistics through interactions with map and charts. This follows the idea of *'Project 1: Statistics Portal'* from the course project description. The website includes rental-price, population, and employment rate in each municipality. Its purpose is to make statistical information easier to explore and compare.

The application is built with HTML, CSS and vanilla JavaScript, following the course requirement to avoid large JavaScript frameworks.

Github link:https://github.com/phamduchaianh/nordwise

Demo link:https://phamduchaianh.github.io/nordwise

2. Technologies and Implementation

- HTML5 and CSS: Page structure, styling and layouts.
- Javascript: Application logic, API requests, Data processing, components management.
- Leaflet: Interactive map and city selections.
- Frappe Charts: bar/line charts visualisation for fetched data.

The application fetches data from external sources, processes municipality identifiers to match records between datasets, and presents the information through the map and charts. Selecting a municipality updates the relevant information and historical visualization.

3. Main features and user manual

Web application step-by-step interaction:

1. Open the application in a web browser and wait for the statistical data to load.
2. Explore the interactive map of Finland and select a municipality.
3. View the selected municipality's data in the information panel.
4. Examine the rental-price comparison chart to identify municipalities with relatively low rental prices.
5. Use the historical chart to explore rental-price changes over time for the selected municipality.
6. Select another municipality on the map to update the displayed information and historical chart.

Note: some statistics may be unavailable for particular municipalities or periods due to missing source data from the API.

4. Self-assessment and requested points

Feature	**Points**	**Justification**
Drag'n'drop new data to charts/maps (there are different data items on screen that user can drag and drop as she wishes)	4/4	Data chips can be dragged onto map/chart to change its layer.
The application show relevant data on a map and user has chance to change the data	3/3	Map has colored data for each areas based on implemented logic. Users can change which layer they want to see from the map.
The application show relevant data on a chart and user has a chance to change the data	3/3	The chart shows the dataset and users can also change which dataset they want to see (lowest/highest and how many municipalities). Rent history chart can switch between line and bar.
User is able to switch between different layers of data on map	2/2	The map has three data layers

		(population, rent, employment rate) and also user-made layers. The map colours and legend update when the layer changes.
By clicking the map user has an option to get to additional charts covering that area	3/4	Clicking a municipality shows its info and a quarterly rent history chart (line/bar). Only rent has history.
There are more than one item of data available (e.g. elections data, employment rate and number of residents) – this means that there are two API calls made	3/3	3 API items were called including Rent, Population, Employment rate.
There are more than two items of data available (e.g. elections data, employment rate and number of residents) – this means that there are three API calls made	2/2	
Data is combined and merged to generate new data, which is then visualized	2/3	User picks two datasets and an operation (+, −, ×, ÷); the result becomes a new chip shown on map and chart. Not saved after reload.
Users can define what should be done to different data items (e.g. values are added, multiplied together etc. before visualization)	2/2	The user chooses the calculation (+, −, ×, ÷) and the number before the data is visualized.
Able to download the visualization as a PNG (or SVG) image	2/2	Both charts have a “Download SVG” button. The map cannot be downloaded.
Well written PDF report	2/3	
Application is responsive and can be used on both desktop and mobile environment	3/4	Layout adapts to small screens; tap replaces drag and drop. Map is small on phones.
Application works on Firefox, Safari, Edge and Chrome	3/3	Tested on Chrome, Edge, Firefox; Safari tested on an iPhone.
The application has clear directory structure and everything is organized well	2/2	JS and CSS folders were created and each file has their own functions called in app.js
Total	**36/40**	

5. AI Usage declaration.

I used **Claude** (Anthropic) and **ChatGPT** (OpenAI).
Claude wrote most of the code for the map layers and legend, drag and drop data chips, chart controls, the employment API call and the SVG download. Claude also wrote most of the CSS, fixed small bugs, explained the code, wrote the README and proofread this report. I pasted the code into the project and edited parts of it. ChatGPT was used for document grammar checking, points checking.
I built the initial version of the app myself and decided how the project is organised into files. I read, tested and edited all the AI-generated code, and I connected it to my own code, so I understand how the data flows from the API to the map and charts. I remain responsible for the submitted application and its functionality.